rogii-wellbore-geology-prediction

26th Place Solution - Stacked UNet + Synthetic Well Pretraining

Rank	26
Team	Tucker Arrants
Author	Tucker Arrants
Topic ID	733136
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733136
Retrieved with Kaggle CLI on 2026-08-06.

Thank you to the hosts and everyone that competed. This was a challenging competition and I was unfortunately unable to stay in the gold zone after the shakeup, but I learned a lot the last few months and look forward to reading other peoples solutions.

TLDR

My approach treats each well as an image: rows are bucketed TVT depths, columns are bucketed MD steps. The vertical axis is a fixed 256 ft window centered on the last known TVT (the projection-start anchor), split into H bins of 256/H ft each (256×1.0 ft for the main models, 224×1.14 ft for one diversity member). The horizontal axis is the well's entire MD range split into W = 2H equal columns, so one column aggregates a handful of consecutive log rows. A stacked UNet consumes this image and outputs, for every MD column, a softmax distribution over the depth rows, trained with cross-entropy against the binned true TVT. The final prediction is the expected value of that distribution (soft-argmax), which gives sub-bin, continuous depth from a classification head - one forward pass predicts the entire well.

The 5 input channels:

- Typewell GR - the typewell log interpolated onto the row grid, so each row carries "the GR the stratigraphy says lives at this TVT." Constant across columns; z-scored per well.
- Horizontal GR - the lateral's GR averaged per MD column, so each column carries "the GR the bit actually measured here." Constant down rows; z-scored per well.
- Hint channel - the known pre-projection-start trajectory rasterized as a one-hot curve (pixel lit at the true depth row for each labeled column, zeros after projection start). This anchors the prediction and turns the task into continuing a visible curve into the unlabeled region rather than finding it from scratch.
- Cost volume - the squared difference between channels 1 and 2 at every (row, column): a full correlation surface where dark valleys mean "the GR measured at this MD matches the GR the typewell expects at this depth." This is the classical geosteering cross-correlation panel, handed to the network as a feature instead of decoded by hand.
- dZ channel - per-column average dZ/dMD (the borehole's local vertical gradient / inclination proxy), z-scored and clipped, constant down rows. Tells the network how steeply the well itself is moving through the frame.

Intuitively: channel 4 paints where the well could be, channel 3 pins where it was, channels 1-2 are the raw ingredients so the net can second-guess the hand-built cost surface, and channel 5 supplies the trajectory physics. The model's job reduces to tracking the plausible ridge of the cost volume forward from the hint, which is why a segmentation-style backbone with a per-column classification head fits the task naturally.

Model Specifications

I started with a standard ImageNet-pretrained ResNet34 from SMP. When I hit a wall around 7.5 CV, I started experimenting with larger backbones. I went straight to the EffNet B7/B8 to test if capacity alone could increase performance - it did. Then I went to an EffNetV2 Large, then the XL, and lastly the EffNet L2, which is the largest EffNet member - when stacked, it was just under 1 billion parameters. The recipe that allowed me to do this was: larger backbone -> bigger pretrain size -> heavier augmentations. The actual performance increases plateaued with the XL, but I trained an L2 purely for diversity, as the L and XL are too correlated. The

B7/B8s were not decorrelated enough to compensate for their higher CV (around 5.8), so they were dropped from the final ensemble.

Other than that, the model was the standard SMP UNet class, with the decoder channels expanded from (256, 128, 64, 32, 16) to (512, 256, 128, 64, 32) - the rest are the default settings.

Stacking / Re-feeding

With this trend of increased capacity netting lower CV, I decided to try a stacked UNet approach. Stage 1 sees the 5-channel image and produces a first per-column depth distribution. Stage 2 sees the same 5 channels plus stage 1's softened output as a 6th channel - a "belief plane" painting where stage 1 thinks the trajectory runs - and produces the refined answer. The handoff is softened with a temperature (T=1.5) and the gradients between the two UNets are detached (allowing gradient flow led to overfitting), so stage 1 is trained by its own cross-entropy loss on the same target (at half weight) - stage 2's loss never touches it. Each stage independently learns to predict the trajectory; stage 2 just gets to see stage 1's opinion while doing so. I got this stacked UNet idea from reading Ruby's solution to the Yale/UNC-CH competition.

The interesting part is inference. Since stage 2 was trained to consume a belief plane, you can feed it its own output and run it again: predict, soften, re-feed, repeat. So, I checked to see if I could re-feed k times and get further improvements - it did at k=2. To push this further, during training, the model sometimes ran multiple rounds in-graph (with probability 0.5) precisely so it would learn to refine multiple rounds of beliefs, with the hope being I could push k higher - it worked to k=3, with performance degrading beyond that.

This self-refeed lets the model reconcile its trajectory estimate with the GR evidence again. This mechanism worked for the EffNetV2 L and XL models, with the optimum at k=3 refeeds. For the L2, refeed was actively worse, with the model quickly overfitting with each k>1.

Pretraining on Synthetic Wells

Real training data is only 773 wells — small for a standard pretrained CNN. So every model is first pretrained on synthetic wells, then finetuned on real ones. The first versions of the UNet model (ResNet34) trained jointly with synthetic wells at a lower loss weight, but that did not fly with the larger models, and pretraining ended up being the best approach. It also saved time, as I could run multiple finetune experiments on the same pretrained models.

A synthetic well starts from a real typewell column: the stratigraphy is borrowed, then a fake lateral is drilled through it — a generated trajectory wandering up and down through the layers, plus geological events (faults, unconformities), measurement noise, and calibration drift. Rendering that through the same image pipeline gives unlimited (well image, true depth curve) pairs.

I learned via brute force experiementation that synthetic data should not imitate real data. I repeatedly tried to close statistical gaps between synthetic and real wells, and it made things worse almost every time. What worked was a curriculum reading: harder-than-real on the geology (big fault throws, aggressive trajectories) and cleaner-than-real on the matching signal (crisp GR correspondence between lateral and typewell). The synthetic world's job is to teach the skill, not to pass a statistical test against reality.

The synthetic pool is generated per fold from that fold's training typewells/geology only, so validation wells are unseen even at pretraining; the deployed models use a pool from all training wells, applying the same rule

at test scale. I am not sure if this is strictly necessary, but I played it safe at the cost of additional pretraining. Furthermore, each seed was given its own pretraining phase, to avoid "getting lucky" pretrained weights that were then inherited by each finetune phase.

Augmentations

Real wells are scarce, so the finetune phase leans hard on augmentation. All of it operates on the raw well before image rendering, so the rendered image stays self-consistent:

- MD warp: smoothly stretch/compress the well along its length, so the same geology appears at different lateral scales.
- Bed warp: smoothly perturb bed thicknesses in the typewell, so the model can't memorize exact layer spacings.
- Horizontal flip: reverse the well direction (with the dZ channel's sign flipped to stay physical).
- Hint drop: randomly delete parts of the known-trajectory hint, so the model can't lean entirely on the anchor and must keep reading the GR correlation.
- Anchor jitter: randomly offset the vertical window relative to the last known TVT, so the model can't learn that the trajectory always starts at the center row and must locate the anchor from the hint channel instead.

Training also uses an EMA of the weights (decay 0.997) as the actual checkpoint - the raw weights are noisy; the averaged copy is consistently better. I also used a small batch size of 8 to help with regularization.

Seed Noise

From the start, I noticed a wide range of CV results with different seeds and generally volatile training. Pretraining, EMA, and augmentations tightened up the per-seed spread, but in the end, I elected to ensemble across 7 seeds per fold. 5 seeds was where the actual seed ensemble boost plateaued, but I elected to add another 2 for peace of mind. I typically had anywhere from 0.2-0.6 ft of ensemble boost compared to the mean across seeds, depending on the variance of seed models across that particular fold. EffNets also helped - I tried many other larger backbones and could not get healthy training from them, so I settled on staying in the EffNet family.

CV Results

My CV was grouped by well, stratified by bucketed TVT range, to try to balance fold difficulty. TVT range (how far a well strays from its anchor) is a strong predictor of how hard a well is, and with 773 wells across 5 folds, an unstratified draw can pile the hard wells into one fold and make scores swing on luck rather than model changes.

I claimed at one point on the discussion board that you could achieve sub-5 ft RMSE with the right model. When I said this, my best model CV was 4.98 - however, there was one fold where the results looked a bit lucky to me, so I retrained it and settled on the "fairer" looking weights, which raised my CV to 5.136, which is the lowest I was able to get any single model. I have never dealt with this "luck" factor before, so am unsure if this was a principled move.

Final Ensemble

Despite the larger-capacity UNets, inference was relatively quick with this "well image" approach, so I was able to ensemble many UNets.

- Model 1: EffNetV2 L, k=3, 5 folds with 7 seeds per fold, with CV 5.157
- Model 2: EffNetV2 L (smaller image), k=3, 5 folds with 7 seeds per fold, with CV 5.252
- Model 3: EffNetV2 XL, k=3, 5 folds with 7 seeds per fold, with CV 5.136
- Model 4: EffNet L2, k=1, 5 folds with 7 seeds per fold, with CV 5.26

The final blend was an even-weight ensemble, as the optimized weights offered a marginal improvement and I did not want to overfit seed noise. The final CV was 5.038 and it scored 6.66 on the private LB.

During the last week, I tried to train a GRU but could not get below CV 7.8. Interestingly, the same binning of TVT / MD + CE loss was much better than direct regression with Huber loss, and the stacked approach also helped. In the end, I abandoned this approach, so the final ensemble is not particularly decorrelated - I am sure it could have been pushed further with more competitive non-UNet models.

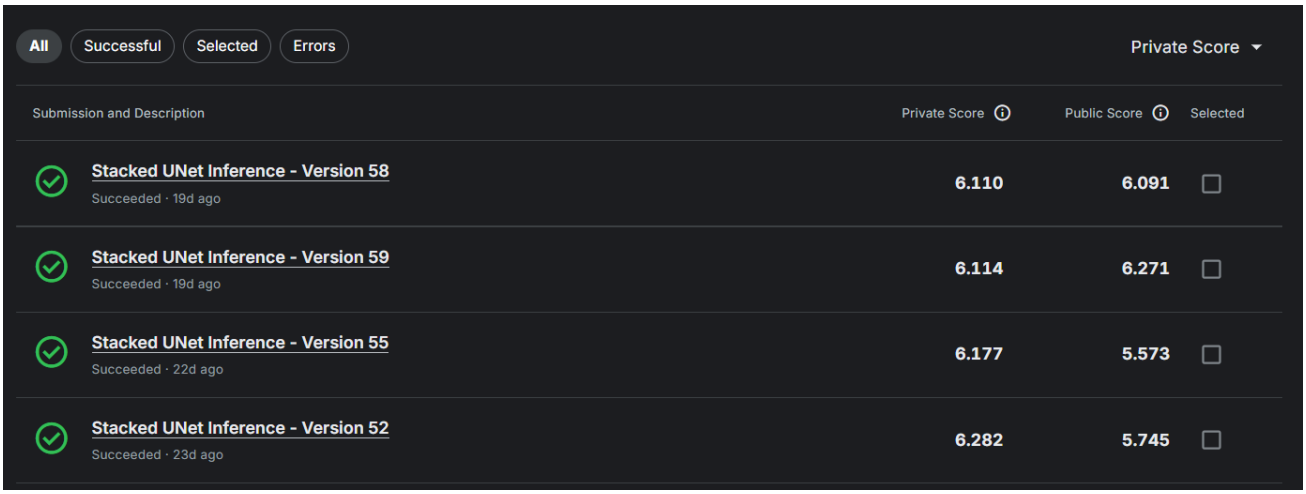
What Did Not Work / Was Unused

- The formation top data was never used, nor were the X and Y coordinates
- BAN-style self distillation
- Other backbones outside the EfficientNet family
- Any auxiliary training tasks
- Any loss modifications beyond vanilla CE (soft CE, MD-weighted CE, etc.)
- Postprocessing/TTA
- PixelShuffle

Unselected Gold Submission Using Neighbor Well Data

A few weeks ago, I built a spatial-neighbor prior based UNet. Every training well's post-PS drift was fit as piecewise-linear slopes over 16 segments, converted from apparent dip (along that well's heading) to true structural dip via azimuth normalization, and poured into a map-wide drift field. For a new well, local-linear regression over the ~50 nearest field samples estimated the true dip at each of its segments, projected back onto its own heading, and integrated from its pre-PS anchor - an 11.8 ft standalone prior. This idea was completely motivated by this awesome notebook here (thank you for sharing).

It entered the model as a 7th input channel onto to the stage 2 model (a Gaussian belief bump the network learned to gate), and it produced my best single model CV of the competition: 5.027. I retired it at deploy because I thought I had CV leakage, as my fold split was not spatial based…tough lesson. The screenshotted submissions below were all using this neighbor spatial data with various single models.



Links: Synth generator for pretraining

Supplemental Q&A

> **MarkDjadchenko · 2026-08-06T00:56:36.653000**
>
> Q: Did you take the literalness option with the segmentation or did you process it somehow in the first versions of the model

> **Tucker Arrants · 2026-08-06T01:01:56.890000**
>
> The UNet head was always a per-MD-column cross-entropy over the depth rows (one class per row), not a binary mask with BCE, and the decode was always the expected value of each column's softmax (soft-argmax), never the argmax (I assume that is what you mean by literalness with segmentation).
>
> The model is actually the exact same one I had after the first couple weeks of the competition, prelimary results shared here, I just scaled up the pretraining, model, and augmentations and the other generic hyper parameter tuning.

> **MarkDjadchenko · 2026-08-06T01:32:00.780000**
>
> A couple more questions: I trained a similar model but only provided a cost map in the form of abs(GR Typewell - Gr Horizontal), and my U-Net achieved 14 LB/CV.
>
> Based on your experience, I'd like to understand whether the augmentations were important, or whether all 5 channels—as in your case—were key.

> **Tucker Arrants · 2026-08-06T01:34:24.967000**
>
> Yes all 5 channels were key. Without the dZ channel, so solely using GR matching channels, it scored around 11.5 CV.
>
> The pretraining was also very important alongside with the augmentations, but I would say the most important part was the pretraining. Just making changes to the synthetic pool and increasing its size shaved a couple feet off my CV.

> **Universal Function Approximator · 2026-08-06T01:50:09.823000**

> Nice job! You mentioned pretraining your U-Net on synthetic well data. Were you doing this all in Kaggle notebooks or were you using your own personal compute environment?

> **Tucker Arrants · 2026-08-06T01:53:39**
>
> I had to dish out quite a bit to rent RTX 6000s on Colab. The L2 model took 3 hours for a single fold, seed model which was brutal.
>
> That being said, I am sure there are more GPU efficient ways of training. The difference between 15k pretraining and 25k pretraining was around 0.2 CV which is small given the near 2x in pretraining time.

> **萤火2018 · 2026-08-06T02:44:49.107000**
>
> Could you share more details about the well synthesis code?My synthesis approach doesn't go as far as full synthesis and pre-training. My method is as follows: for each training well, I generate `n_synth` synthetic versions. All other features remain constant, and I only modify `h_gr` by setting `h_gr = h_gr_sim + h_gr_noise` (where `gr_noise` is derived from other random training wells), then add this as new data for training. Here, `gr_noise = gr_obs - gr_sim`.

> **Tucker Arrants · 2026-08-06T03:00:47.163000**
>
> Of course, here you are - told Claude to strip it out from my local training code, so it might be buggy but it has all the details. I am sure it is far from optimal, as it was extremely time consuming to "tune" the pool, so I only ran a handful of targetted experiments with it. The settings in that notebook are what I used for my final models.
>
> https://www.kaggle.com/code/tuckerarrants/rogii-synth-generator?scriptVersionId=340488304